

HW_2

이규환 2026406017

카메라 영상에서 중앙의 흰 선과 파란 선을 기준으로 주황색 콘과 초록색 콘이 각각 어느 위치에 있는지를 실시간으로 판별하는 프로그램을 Qt GUI로 만들었다. 위치는 4개의 사분면, 두 선 위의 네 경우, 중앙, 콘이 보이지 않는 경우까지 10가지로 구분하고, 인식된 콘은 바운딩박스 내부만 잘라 별도 이미지로 표시한다. 패키지 이름은 cv_hw2, 실행은 `ros2 run cv_hw2 cv_hw2` 이다.

1. OpenCV 개념 정리

1-1. 픽셀과 색 표현 방식

이미지는 컴퓨터에게는 숫자가 격자로 늘어선 배열이고 격자 한 칸이 픽셀이다. 640×480 컬러 영상은 한 프레임이 92만 개가 넘는 숫자를 들고 있고 이걸 초당 30번 처리해야 실시간이 되므로, 필터 커널 크기를 정할 때 연산량을 신경 쓰게 됐다. OpenCV에서 이 배열을 담은 자료형이 `cv::Mat`이다.

표현 방식	채널 수	값의 범위	픽셀 하나가 들고 있는 것
RGB	3	각 0 ~ 255	빨강, 초록, 파랑의 세기
Grayscale	1	0 ~ 255	밝기 하나
HSV	3	H 0~179, S·V 0~255	색조, 채도, 명도
Binary	1	0 또는 255	검정이나 흰색이나

처음에 헷갈렸던 것이 두 가지 있다. **OpenCV의 채널 순서는 RGB가 아니라 BGR**이고, **HSV의 H는 0~179**다. 색상환이 0~360도인데 한 바이트에 넣느라 절반으로 줄여 쓰기 때문이다.

1-2. 왜 RGB 가 아니라 HSV 인가

조명이 밝아지면 R, G, B가 전부 같이 올라가서 RGB에서는 "무슨 색인가"와 "얼마나 밝은가"를 떼어낼 수 없다. HSV는 이 둘을 분리해 놓았다. **H(색조)**는 색상환 위의 각도로 조명이 바뀌어도 잘 변하지 않고, **S(채도)**는 색이 얼마나 진한지, **V(명도)**는 얼마

나 밝은지를 나타낸다. 그래서 기본 전략은 **H로 색 계열을 고르고 S와 V로 흐리거나 어두운 것을 걸러내는** 것이 된다.

1-3. 이진화와 inRange

이진화는 픽셀을 검은색(0)과 흰색(255)으로만 표현하는 것이다. 찾으려는 대상만 흰색으로 남기면 이후 처리는 "흰 영역이 어디에 얼마나 있는가"만 보면 된다. OpenCV에서는 inRange로 특정 색 범위만 뽑아 이진화한다.

```
cvtColor(img, img_hsv, COLOR_BGR2HSV);  
inRange(img_hsv, Scalar(h_low, s_low, v_low),  
        Scalar(h_high, s_high, v_high), mask);
```

가장 중요한 건 **세 조건이 AND로 묶인다는** 점이다. H가 아무리 정확해도 S가 하한보다 낮으면 그 픽셀은 버려진다. "H는 맞았는데 화면이 전부 까맣다"는 상황의 범인은 거의 항상 S나 V의 하한이었다.

1-4. 가우시안 필터와 모폴로지

어떤 픽셀이 주변과 극단적으로 값이 다르면 그게 노이즈다. 가우시안 필터는 가운데에 가중치를 크게 주고 멀어질수록 작게 주는 커널로 주변 값을 섞으며, 가중치 합이 1이라 전체 밝기는 유지된다. HW1에서 흰색 덩어리가 필터 없이 17개에서 5 × 5 적용 후 4개로 줄어드는 것을 확인해 이번에는 5 × 5를 항상 적용한다.

모폴로지는 이진화 **후에** 결과를 다듬는 연산이다. 가우시안이 원본을 다듬는 것과 역할이 다르다.

연산	동작	효과
침식 (erode)	객체 외곽을 깎아낸다	작은 점이 사라진다
팽창 (dilate)	객체 외곽을 확장한다	작은 구멍이 메워진다
열기 (open)	침식 후 팽창	작은 노이즈 제거에 효과적
닫기 (close)	팽창 후 침식	내부의 빈 구멍을 없앤다

이번 과제에서는 **열기 다음에 닫기** 순서로 적용했다. 열기로 배경의 점 노이즈를 먼저 지우고 닫기로 콘 표면의 반사광 구멍을 메우기 위해서다. 순서를 반대로 하면 닫기가 노이즈 점들을 이어붙여 더 큰 덩어리를 만들 수 있다.

```
Mat kernel = getStructuringElement(MORPH_RECT, Size(5, 5));  
morphologyEx(mask, mask, MORPH_OPEN, kernel);
```

```
morphologyEx(mask, mask, MORPH_CLOSE, kernel);
```

2. 과제 분석과 설계

2-1. 판별해야 하는 10 가지 경우

구분	경우	판정 조건
사분면	1 / 2 / 3 / 4 사분면	흰선·파란선 중심 기준 상하좌우
흰선 위	위쪽 / 아래쪽 하얀선 위	콘 중심 x가 흰선의 x 범위 안
파란선 위	왼쪽 / 오른쪽 파란선 위	콘 중심 y가 파란선의 y 범위 안
중앙	중앙	두 조건을 동시에 만족
없음	존재하지 않음	콘 마스크 면적이 기준 미만

설계상 중요한 판단이 하나 있었다. "선 위"와 "사분면"을 가르려면 선을 **면적을 가진 띠**로 봐야 한다. 선을 좌표 하나로 보면 콘 중심이 정확히 그 좌표에 놓일 확률이 거의 0이라 "선 위"가 나오지 않으므로, 선의 바운딩박스 폭과 높이를 그대로 판정 범위로 썼다.

2-2. 전체 파이프라인

```
카메라 프레임 -> GaussianBlur 5x5 -> cvtColor(BGR2HSV)
-> inRange x 4 -> morphologyEx (열기 -> 닫기)
-> countNonZero, boundingRect -> 위치 판별 + 크롭
```

네 대상이 같은 경로를 지나고 HSV 범위만 다르므로, 마스크 생성을 makeMask 함수 하나로 묶고 대상 번호만 바꿔 네 번 돌린다.

2-3. findContours 를 쓰지 않은 이유

보통은 findContours로 외곽선을 찾아 가장 큰 것을 고르지만 교육자료에 없는 함수라 배운 범위에서 풀 방법을 찾았다. 답은 boundingRect였다. **1채널 이진 이미지를 넣으면 0이 아닌 픽셀 전체를 감싸는 사각형을 돌려준다.** 존재 여부는 countNonZero로 흰 픽셀을 세서 기준값(300픽셀)과 비교한다.

```
if (cv::countNonZero(mask) < MIN_AREA)
    return cv::Rect();
return cv::boundingRect(mask);
```

다만 boundingRect는 흰 픽셀 **전부**를 감싸므로 화면 구석에 노이즈 점 하나만 남아도

박스가 화면 전체로 커진다. 그래서 이 설계에서는 **이진화 결과가 깨끗한 것이 선택이 아니라 필수 조건**이 된다. 가우시안과 모폴로지를 반드시 거치게 한 이유, 4장에서 HSV 값을 오래 붙잡은 이유가 전부 여기서 나온다.

2-4. 기준선과 위치 판별

흰 선은 세로로 길어 바운딩박스의 x 범위를, 파란 선은 가로로 길어 y 범위를 쓴다. 파란 선은 흰 선과 콘에 가려 마스크가 서너 조각으로 끊겨도 boundingRect가 전체를 감싸므로 올바른 y 범위가 나온다. 콘 중심이 흰 선의 x 범위 안인지(on_white), 파란 선의 y 범위 안인지(on_blue), 이 두 조합으로 대부분이 갈린다.

on_white	on_blue	추가 조건	결과
O	O	-	중앙
O	X	콘 y < / > 파란선 중심 y	위쪽 / 아래쪽 하얀선 위
X	O	콘 x < / > 흰선 중심 x	왼쪽 / 오른쪽 파란선 위
X	X	오른쪽.위 / 왼쪽.위	1사분면 / 2사분면
X	X	왼쪽.아래 / 오른쪽.아래	3사분면 / 4사분면

여기에 예외 두 가지를 앞에 붙였다. 콘 박스가 비어 있으면 "존재하지 않음", 기준선이 검출되지 않았으면 "기준선 없음"을 반환한다. 기준선이 없을 때 중심 좌표가 0이 되어 모든 콘이 1사분면으로 잘못 판정되는 것을 겪었기 때문인데, 틀린 답보다 판단을 보류하는 편이 낫다고 봤다.

2-5. 바운딩박스 내부 추출

콘이 인식되면 원본 프레임에서 박스 영역만 잘라 새 이미지로 만든다. 여기서 **clone()**을 반드시 붙여야 한다. clone 없이 받으면 원본 프레임의 메모리를 공유해서 다음 프레임이 들어오는 순간 잘라낸 이미지도 같이 바뀐다. 실시간이라 화면이 깜빡이는 것으로 바로 드러났다.

3. 구현

교육자료의 예시 화면을 참고해 처리 단계가 한눈에 보이도록 화면을 6분할했다. 원본, 기준선과 바운딩박스를 그린 결과, 네 대상의 이진화 결과를 각각 띄운다. 중간 결

과를 전부 보여주는 이유는 HSV 값을 맞추는 때 어느 단계에서 잘못됐는지 눈으로 바로 확인하기 위해서고, 실제로 4장의 문제들을 이 화면 덕분에 찾아냈다. 슬라이더는 여섯 개인데 조절할 대상은 네 개라 값을 2차원 배열에 담고 라디오 버튼으로 대상을 바꿀 때마다 불러온다.

카메라는 /dev 폴더에서 video로 시작하는 장치를 찾는 다음 각각을 VideoCapture로 열어보고 실제로 열리는 것만 목록에 넣는다. 리눅스에서는 카메라 하나에 video 노드가 두 개씩 생기는데 그중 하나는 영상 캡처용이 아니기 때문이다. 프레임은 QTimer를 33밀리초 주기로 읽어 초당 약 30프레임이 된다. 교육자료 예시가 usb_cam 토픽 방식이라 ROS 토픽 입력도 남겨뒀는데, Image 메시지가 바이트 배열에 크기와 인코딩이 붙은 형태라 cv_bridge 없이 Mat 생성자에 그대로 넘겨 변환했다.

Qt의 exec()와 ROS의 spin()은 둘 다 끝나지 않는 루프라 같이 돌릴 수 없어서 ROS 쪽은 QThread를 상속한 QNode에서 spin_some()으로 돌린다. ROS 콜백은 Qt 메인 스레드가 아닌 별도 스레드에서 실행되므로 콜백에서 위젯을 직접 건드리면 안 된다. 뮤텍스로 잠그고 프레임을 저장한 뒤 시그널만 보내고, 화면 갱신은 메인 스레드에서 한다.

4. HSV 값 튜닝 과정

이번 과제에서 시간을 가장 많이 쓴 부분이다. 코드를 다 짜고도 콘이 잡히지 않아 화면의 픽셀값을 직접 찍어보며 원인을 찾아야 했다.

4-1. 과노출 — 물체의 중심이 하얗게 날아간다

처음 기본값은 콘이 선명한 색이라 가정하고 채도 하한을 높게(초록콘 90, 주황콘 150) 잡았는데 두 콘 모두 이진화 결과가 거의 검은 화면이었다. 픽셀값을 재보니 카메라 노출이 세서 **콘의 중심부가 하얗게 날아가 있었다**. 초록콘 몸통의 채도 중앙값이 35, 주황콘이 116이라 선명한 부분은 얇은 테두리뿐이었고 그마저 모폴로지 열기에 지워졌다. 채도 하한을 20과 70으로 낮추자 콘 전체가 잡혔다. **물체가 원래 무슨 색인지가 아니라 카메라에 찍힌 값을 봐야 한다는 것이 교훈이다**.

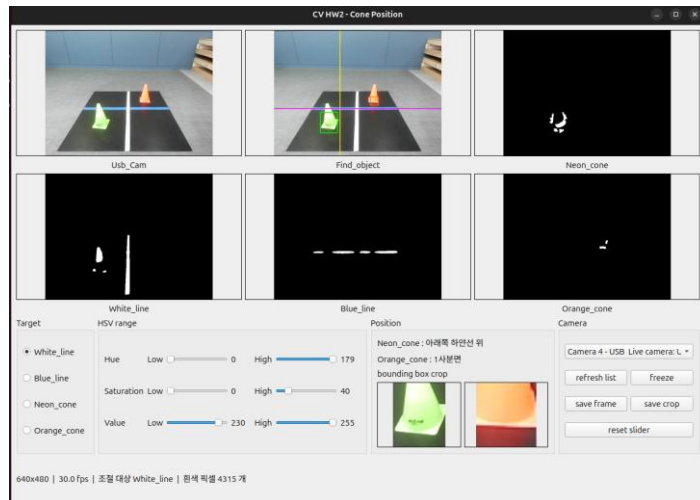


그림 1. 초기 기본값으로 실행한 화면. Neon_cone 마스크에 콘의 테두리만 초승달 모양으로 남았고 Orange_cone 마스크는 거의 비어 있다. White_line 마스크에는 선 외에 초록콘 덩어리가 함께 잡혀 있다.

4-2. 흰 선과 형광 초록콘이 겹치는 문제

과노출은 흰 선 쪽에도 문제를 만들었다. 흰 선은 명도가 높고 채도가 낮은 것으로 찾는데 하얗게 날아간 초록콘 중심부도 똑같았다. 그래서 White_line 마스크에 콘 덩어리가 함께 잡혔고(그림 1), boundingRect 때문에 기준선의 x 범위가 콘 쪽까지 늘어나 콘이 "아래쪽 하얀선 위"로 잘못 판정됐다. 두 영역을 비교해보니 **콘은 아무리 날아가도 초록빛이 남아 채도가 0보다 크고, 흰 선만 정확히 0이었다.** 흰색은 세 채널이 같은 값이라 채도가 정의상 0이 되지만 초록콘은 초록 채널이 먼저 포화되기 때문이다. 채도 상한을 0으로 조이자 흰 선만 남았다.

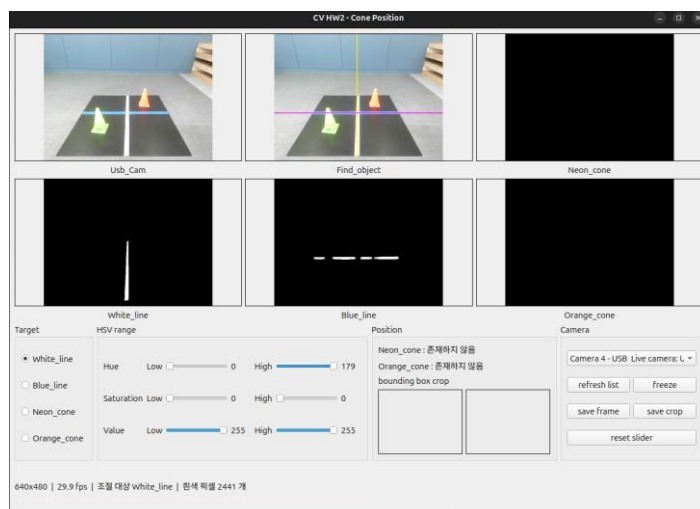


그림 2. 흰 선 임계를 조인 뒤의 화면. 두 기준선은 안정적으로 잡히지만 콘의 채도 하한이 아직 높아 두 콘 모두 "존재하지 않음"으로 나온다.

4-3. 파란 선과 캐비닛, 주황콘과 나무 선반

배경의 파란 회색 캐비닛은 색조가 파란 선과 똑같이 101 근처라 색조로는 구분할 수 없었다. 채도는 선이 94~124, 캐비닛이 67~78이고 명도는 선이 220까지 올라가는 데 비해 캐비닛은 185에서 끝나, 채도 하한 90과 명도 하한 190을 같이 걸자 완전히 걸러졌다.

화면 오른쪽 위의 나무 선반은 주황콘과 가장 비슷했다. 색조는 19 대 22, 채도도 68 대 79라 채도 하한을 올리면 콘까지 사라졌다. 여기서는 명도가 답이었다. 선반의 명도는 98퍼센트 지점이 225에서 끝나는데 주황콘은 과노출 덕분에 251까지 올라가, 명도 하한을 225로 두자 선반은 한 픽셀도 남지 않았다.

4-4. 최종 기본값

대상	Hue	Saturation	Value	핵심 판단 근거
White_line	0 ~ 179	0 ~ 0	250 ~ 255	채도가 정확히 0인 것만
Blue_line	90 ~ 115	90 ~ 255	190 ~ 255	명도로 캐비닛 제외
Neon_cone	28 ~ 55	20 ~ 255	150 ~ 255	채도 하한을 크게 낮춤
Orange_cone	0 ~ 30	70 ~ 255	225 ~ 255	명도로 나무 선반 제외

네 대상을 맞추며 공통적으로 배운 것이 있다. 두 물체가 헛갈릴 때는 세 채널을 전부 조이는 것이 아니라 **가장 잘 갈라지는 채널 하나를 찾아 거기에 기준을 거는 것**이 효과적이다. 색조로 겹치면 채도를, 채도로도 겹치면 명도를 보는 식이다. HSV가 세 채널로 나뉘어 있는 이유를 이 과정에서 몸으로 이해하게 됐다.

5. 실행 결과

카메라를 선택하면 바로 영상이 들어오고 여섯 개 화면이 초당 30프레임으로 갱신된다. Find_object 화면에 기준선이 노란 세로선과 자홍 가로선으로 그려지고 두 콘에 사각형이 표시되며, Position 영역에 판별 결과와 잘라낸 콘 이미지가 나온다. 콘을 옮기면 판별이 프레임마다 따라 바뀌고 화면 밖으로 빼면 "존재하지 않음"이 되면서 크롭

칸도 비워진다. 콘을 선 쪽으로 밀면 중심이 선의 폭 안으로 들어가는 순간 "선 위"로, 두 선이 만나는 지점에서는 "중앙"이 표시돼 선을 따라 본 설계가 의도대로 동작하는 것을 확인했다.

전체 실행 영상은 같은 폴더에 첨부했다. 카메라를 선택하는 것부터 콘을 여러 위치로 옮기며 판별이 바뀌는 모습, 슬라이더로 HSV 값을 조절할 때 이진화 결과가 변하는 모습까지 담았다.

6. 한계와 개선 방향

손이나 소매가 들어오면 기준선이 틀어진다. 밝은 피부나 옷이 흰 선 마스크에 잡히고 boundingRect가 그것까지 감싸면서 콘이 "하얀선 위"로 잘못 표시되는 것을 확인했다. **매트 밖에 놓인 콘도 사분면으로 표시되는** 문제도 있다. 판별이 기준선 위치만 보고 화면 전체를 네 칸으로 나누기 때문이다. **선이 기울어지면 대응할 수 없다**는 한계도 있는데, 이 경우까지 처리하려면 선의 픽셀들로 직선을 근사해 콘 중심에서 그 직선까지의 **부호 있는 거리**로 판단해야 한다.

되돌아보면 튜닝의 어려움은 대부분 **카메라의 자동 노출** 때문이었다. HSV 범위를 극단으로 조여 대응했지만 채도 0, 명도 250 이상이라는 흰 선 조건은 지금의 과노출 상태에서만 성립해서 조명이 어두워지면 기준선을 잃는다. 노출과 화이트밸런스를 v4l2-ctl로 수동 고정해 과노출을 없애면 평범한 HSV 값으로도 잡힌다. 문제를 알고리즘에서 막는 것보다 입력에서 없애는 것이 더 근본적이라는 점을 배웠다.

```
v4l2-ctl -d /dev/video0 --set-ctrl=auto_exposure=1  
v4l2-ctl -d /dev/video0 --set-ctrl=exposure_time_absolute=100
```

7. 정리하며

HSV는 색과 밝기가 채널로 분리되어 있어 조명이 일정하지 않은 환경에서 유리하다. inRange의 세 조건은 AND로 묶이므로 색이 안 잡히면 채도와 명도의 하한을 먼저 의심해야 한다. 두 물체가 헛갈릴 때는 **가장 잘 갈라지는 채널 하나를 찾아 기준을 거는** 것이 효과적이다. 가장 크게 남은 것은 튜닝 과정이다. HSV 값 맞추기가 감으로 슬라이더를 움직이는 일인 줄 알았는데, 실제로는 두 대상의 픽셀값 분포를 보고 갈라지는 채널을 찾는 분석 작업이었다.